HomeCart

Complete Code Walkthrough & Local Development Guide

A practical reference for understanding the codebase, editing features, and running the app on your local machine.

Tech Stack Overview

Layer	Technology	Why
Frontend	React 19 + Tailwind CSS + Shadcn UI	Modern reactive UI, utility-first styling
Backend	FastAPI (Python)	Fast async API, auto-generated docs
Server	Uvicorn (ASGI) on port 8001	Production-grade ASGI server
Database	MongoDB (via Motor — async driver)	Flexible schema, no migrations
Auth	JWT + bcrypt + httpOnly cookies	Secure, stateless authentication
Process Manager	Supervisor	Auto-restarts, manages logs

Project Structure

```
/app/
├── backend/
│   ├── server.py          ← All API endpoints (~495 lines)
│   ├── .env               ← MongoDB URL, JWT secret, admin credentials
│   └── requirements.txt   ← Python dependencies
│
├── frontend/
│   ├── public/index.html  ← Loads Google Fonts (Cabinet Grotesk, Work Sans)
│   ├── src/
│   │   ├── App.js                 ← Routing
│   │   ├── index.css              ← Tailwind + design tokens
│   │   ├── contexts/AuthContext.js ← Login state with httpOnly cookies
│   │   ├── components/
│   │   │   ├── ProtectedRoute.js   ← Route guard
│   │   │   └── ui/                 ← Shadcn components
│   │   ├── pages/
│   │   │   ├── Login.js
│   │   │   ├── Register.js
│   │   │   └── Dashboard.js        ← Main app (~770 lines)
│   ├── tailwind.config.js
│   └── .env                       ← REACT_APP_BACKEND_URL
│
└── memory/
    ├── PRD.md
    └── test_credentials.md
```

Backend Deep Dive — /app/backend/server.py

The backend is one file split into logical sections. It uses async MongoDB (Motor) for non-blocking I/O and bcrypt + JWT for authentication.

1. Auth Helpers

```
def hash_password(password)       # bcrypt hash
def verify_password(plain, hash)  # bcrypt verify
def create_access_token(uid, email)  # JWT, 15-min expiry
def get_current_user(request)     # Reads access_token cookie, validates JWT
```

2. API Endpoints (all prefixed /api)

Endpoint	Purpose
`POST /api/auth/register`	Create user, set httpOnly cookies
`POST /api/auth/login`	Authenticate, issue tokens
`GET /api/auth/me`	Get current user from cookie
`POST /api/homes`	Create household with unique Home ID
`POST /api/homes/join`	Join existing home via ID
`GET /api/homes`	Get user's homes
`POST /api/lists`	Create Daily/Weekly/Monthly list
`POST /api/items`	Add item to list (auto-saves to catalogue)
`GET /api/items/{id}/prices`	Mock price comparison across 6 vendors
`GET /api/lists/{id}/basket`	Consolidated basket cost per vendor

3. The Mock Price Engine (this is what you'll replace)

```
VENDORS = ["Amazon Fresh", "Blinkit", "BigBasket",
           "JioMart", "Zepto", "Swiggy Instamart"]

def calculate_price(item_name, vendor, quantity):
    # Deterministic pseudo-random price for demo
    seed = sum(ord(ch) for ch in item_name + vendor)
    base = 35 + (seed % 250)
    discount = 0.82 if seed % 5 == 0 else 0.9 if seed % 3 == 0 else 1
    return round(base * quantity * discount, 2)
```

Frontend Deep Dive

Data Flow

```
User clicks "Add Item"
  → Dashboard.js calls:
    axios.post(`${API}/items`, {...}, { withCredentials: true })
  → Backend reads access_token cookie via get_current_user()
  → Saves to MongoDB
  → Returns item
  → React re-fetches via loadItems() and loadBasketComparison()
```

Key File: /app/frontend/src/pages/Dashboard.js

This is the entire app UI. It contains:

- State hooks (lines 30-50): homes, lists, items, catalogue, modals
- Data loaders (lines 70-150): loadHomes(), loadItems(), loadBasketComparison()
- Action handlers (lines 150-260): createHome(), addItem(), toggleItemComplete(), shareOnWhatsApp()
- JSX layout (lines 280+): Header → Sidebar → Main → Modals

How to Edit & Add Features

Example 1: Add a new field to items (e.g., 'notes')

Backend (server.py):

```
class AddItemRequest(BaseModel):
    list_id: str
    name: str
    category: str
    quantity: float
    unit: str
    notes: Optional[str] = None  # ← ADD THIS
```

Then in add_item(), include "notes": req.notes in the document.

Frontend (Dashboard.js):

```
const [itemForm, setItemForm] = useState({
  name: '', category: 'Groceries', quantity: 1, unit: 'kg',
  notes: ''  // ← ADD
});

// Add an <input> for notes in the Add Item modal.
```

Example 2: Replace mock prices with real scraping

Step 1: Install scraping library

```
cd /app/backend
pip install httpx beautifulsoup4
pip freeze > requirements.txt
```

Step 2: Create /app/backend/price_providers.py

```
import httpx
from bs4 import BeautifulSoup

async def fetch_blinkit_price(item_name: str):
    async with httpx.AsyncClient(timeout=10.0) as client:
        response = await client.get(
            f"https://blinkit.com/search?q={item_name}",
            headers={"User-Agent": "Mozilla/5.0..."}
        )
        # Parse HTML and extract price
        soup = BeautifulSoup(response.text, 'html.parser')
        # ... extract price ...
        return {"vendor": "Blinkit", "price": parsed_price, "url": url}
```

Step 3: Update /api/items/{id}/prices in server.py

```
from price_providers import fetch_blinkit_price, fetch_zepto_price
import asyncio

@api_router.get("/items/{item_id}/prices")
async def get_item_prices(item_id: str, request: Request):
    user = await get_current_user(request)
    item = await db.list_items.find_one({"_id": ObjectId(item_id)})

    # Run all 6 scrapers in parallel
    results = await asyncio.gather(
        fetch_blinkit_price(item["name"]),
        fetch_zepto_price(item["name"]),
        # ... etc
        return_exceptions=True
    )
    prices = [r for r in results if not isinstance(r, Exception)]

    # Cache for 1 hour to reduce scraping load
    await db.price_cache.update_one(
        {"item_name": item["name"]},
        {"$set": {"prices": prices,
                  "cached_at": datetime.now(timezone.utc)}},
        upsert=True
    )

    min_price = min(p["price"] for p in prices)
    for p in prices:
        p["best"] = p["price"] == min_price
    return prices
```

Reality check: Most Indian quick-commerce apps don't offer public APIs. Practical paths:

- **RapidAPI** marketplace — search 'grocery price comparison India'
- **BrightData / ScraperAPI** — paid scraping services with anti-bot bypass
- **Amazon Product Advertising API** — official & legal for Amazon Fresh

Example 3: Add a 4th category (e.g., 'Snacks')

Frontend (Dashboard.js):

```
const CATEGORIES = [
  { id: 'Groceries', icon: Grains, color: '#1A3626', img: '...' },
  { id: 'Vegetables', icon: Carrot, color: '#2D6A4F', img: '...' },
  { id: 'Medicines', icon: Pill, color: '#FF6B35', img: '...' },
  { id: 'Snacks', icon: Cookie, color: '#FFB703', img: 'URL' },  // ← ADD
];
```

That's it — the UI auto-iterates over CATEGORIES, so the new section appears everywhere automatically.

Running Locally to Test Deployment

Your app is already running in the cloud environment via Supervisor. To run on your own machine:

Prerequisites

Node.js 18+, Yarn, Python 3.11+, MongoDB

Step 1: Get the code

```
# Push to GitHub from this environment, then clone:
git clone <your-repo>
cd <your-repo>
```

Step 2: Start MongoDB

```
# Install MongoDB Community Edition, then:
mongod --dbpath ~/mongodb-data
```

Step 3: Backend setup

```
cd backend
python -m venv venv
source venv/bin/activate    # Windows: venv\Scripts\activate
pip install -r requirements.txt

# Edit .env (keep these defaults for local):
# MONGO_URL="mongodb://localhost:27017"
# DB_NAME="homecart"
# JWT_SECRET="any-long-random-string"
# FRONTEND_URL="http://localhost:3000"

uvicorn server:app --host 0.0.0.0 --port 8001 --reload
```

Backend now running at http://localhost:8001. Visit http://localhost:8001/docs for auto-generated Swagger UI to test all endpoints interactively.

Step 4: Frontend setup (new terminal)

```
cd frontend
yarn install

# Edit .env:
# REACT_APP_BACKEND_URL=http://localhost:8001

yarn start
```

Frontend opens at http://localhost:3000

Step 5: Test

- Visit http://localhost:3000
- Register a new user OR login with admin@homecart.com / admin123
- Create a home → create a list → add items → click 'Compare prices'

Useful Commands While Developing

Command	Purpose

`sudo supervisorctl restart backend`	Restart FastAPI after .env changes
`sudo supervisorctl status`	Check what's running
`tail -f /var/log/supervisor/backend.err.log`	Watch backend errors live
`tail -f /var/log/supervisor/frontend.out.log`	Watch React compile output
`curl http://localhost:8001/api/auth/me`	Test API directly
`Visit http://localhost:8001/docs`	Interactive Swagger UI

Recommended Next Steps

- **Easy wins**: Add the 'notes' field per item, add Snacks category
- **Medium**: Build a real price_providers.py module starting with Amazon Product Advertising API (only one with official, legal API access)
- **Big**: Use RapidAPI's grocery comparison APIs for Blinkit/Zepto/etc. (paid but production-ready)

Want help implementing any of these? Just ask in the chat — Emergent will wire it up for you.